SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: *Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Industry Problem Solving Task

Code of Conduct:

I Cheredla Jasvina / 192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	

Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	

Annexure A

Industry Problem Solving Task

1: Automated Defect Detection in Manufacturing
A precision engineering company wants to automatically detect defects in mechanical components using computer vision. Parts vary slightly in shape due to manufacturing tolerances.
Questions:

1. **How can PCA-based shape priors and contour-based representations be used to model standard component shapes?**

Answer :

- PCA-Based Shape Priors
- Principal Component Analysis (PCA) can be used to learn the normal shape variation of mechanical components.
- Collect images of many non-defective components.
- Segment each component from its background.
- Extract shape information such as boundary points, contours, or shape descriptors.
- Align all shapes to a common position and scale.
- Convert each shape into a numerical vector.
- Apply PCA to reduce the dimensionality and identify the major modes of shape variation.

The PCA model can be represented as:
where:

- **= observed component shape**
- **= average normal shape**
- **= principal component matrix**
- **= PCA coefficients**

A new component is compared with the learned normal shape. If its shape differs significantly from the allowed PCA variation, it can be considered potentially defective.
Contour-Based Representation
The component boundary can be extracted using edge detection and contour detection.
Useful contour features include:

- Area

- Perimeter
- Aspect ratio
- Circularity
- Convexity
- Hu moments
- Curvature
- Contour-to-template distance

For example, the circularity measure is:

where is area and is perimeter.

A normal component should have a contour close to the expected contour. Missing material, cracks, dents, or deformation will produce noticeable contour differences.

2. Propose a pipeline using machine learning (SVM, GMM) or deep learning (CNN) to classify defective vs. non-defective parts.

Answer :

Step 1: Image Acquisition

Capture images of mechanical components using an industrial camera under controlled conditions.

Dataset:

- Normal parts
- Defective parts
- Different manufacturing tolerances
- Different defect types

Step 2: Preprocessing

Apply:

- Noise filtering
- Image resizing
- Contrast enhancement
- Background removal
- Intensity normalization

Step 3: Segmentation

Separate the mechanical component from the background using:

- Thresholding
- Edge detection
- Contour extraction
- Morphological operations

Step 4: Feature Extraction

Two approaches can be used.

Traditional ML:

Extract:

- PCA shape coefficients
- Contour features
- Area

- Perimeter
- Texture features
- Edge features

These features are given to an SVM or GMM.

SVM:

The SVM learns a decision boundary between:

For non-linear defect patterns, an RBF kernel can be used.

GMM:

A Gaussian Mixture Model learns the probability distribution of normal

$$P(x)=k=1\sum K\pi kN(x|\mu k,\Sigma k)$$

components:

If a new component has a very low probability under the learned normal distribution, it can be classified as defective.

CNN-Based Approach

A pretrained CNN such as ResNet or MobileNet can be fine-tuned using industrial component images.

Step 5: Evaluation

Use:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

For manufacturing, recall for defective parts is especially important because missing a defective component can lead to safety and quality problems.

3. **Discuss methods to handle noise, partial occlusions, and varying illumination in industrial images.**

Answer :

A. Noise

Industrial images may contain sensor noise, dust, or small unwanted objects.

Methods:

- Gaussian filtering
- Median filtering
- Bilateral filtering
- Morphological opening and closing
- Image denoising

A median filter is particularly useful for removing salt-and-pepper noise while preserving edges.

B. Partial Occlusions

A component may be partially covered by another object or fixture.

Possible solutions:

- Use multiple camera views.

- Use contour fragments instead of requiring the complete contour.
- Use local features that remain visible during occlusion.
- Train CNN models using partially occluded training images.
- Use data augmentation such as random cropping and masking.
- Use PCA shape priors to estimate the missing portion of the expected shape.
- Thus, even when part of the component is hidden, the visible region can still be compared with the expected normal shape.

C. Varying Illumination

Changes in lighting can affect edges, texture, and pixel intensity.

Methods include:

- Controlled industrial lighting
- Uniform LED illumination
- Histogram equalization
- CLAHE
- Gamma correction
- Intensity normalization
- Image augmentation with different brightness levels

For example, CLAHE improves local contrast while reducing the effect of uneven illumination.

Overall Proposed System

This system can automatically identify defects such as cracks, dents, scratches, missing material, dimensional deviations, and abnormal shapes, while PCA and contour representations help tolerate the small shape variations caused by normal manufacturing tolerances.

2: Autonomous Vehicle Obstacle Recognition

A self-driving car company needs to detect rare or unexpected obstacles on urban roads. The labeled data for these obstacles is limited.

Questions:

1. **Explain how data augmentation using GANs can help generate synthetic images for training.**

Answer :

Generative Adversarial Networks (GANs) can generate realistic synthetic images of rare or unexpected obstacles when only a small amount of labeled training data is available.

A GAN consists of two networks:

- **Generator (G):** Creates synthetic obstacle images from random noise.
- **Discriminator (D):** Determines whether an image is real or generated.

The two networks are trained against each other:

As training progresses, the generator learns to create increasingly realistic images.

Steps:

1. Collect available real images of obstacles.
2. Train a GAN using these images.

3. **Generate synthetic images of rare obstacles such as:**
   - Fallen objects
   - Unusual vehicles
   - Road debris
   - Animals
   - Damaged traffic signs
4. Combine real and synthetic images.
5. Train the obstacle-detection model using the expanded dataset.
6. Validate the model using real-world images.

GANs can also generate variations in:

- Lighting
- Weather
- Camera angle
- Object size
- Background
- Object position

Therefore, GAN-based augmentation increases dataset diversity and helps the model recognize rare obstacles that are difficult to collect in real-world driving conditions.

2. Compare the effectiveness of CNNs versus Vision Transformers for detecting small or sparse objects in high-resolution images.

Answer :

Feature	**CNN**	**Vision Transformer (ViT)**
Basic operation	Convolution	Self-attention
Local feature detection	Excellent	Good
Small-object detection	Generally effective	Can be effective with suitable architecture
Global context	Limited initially	Excellent
High-resolution images	Computationally efficient with optimized CNNs	Computationally expensive
Training data requirement	Lower	Usually higher
Embeddeddeploymen	Easier	More difficult

Feature	CNN	Vision Transformer (ViT)
Computational cost	Lower	Higher
Long-range relationships	Limited	Excellent

CNN

CNNs use convolution filters to detect local features such as:

- Edges
- Shapes
- Textures
- Small objects

For autonomous vehicles, architectures such as lightweight YOLO, MobileNet, or EfficientDet-style detectors can provide fast inference.

CNNs are particularly suitable for embedded systems because convolutions can be highly optimized using GPUs, NPUs, and other accelerators.

Vision Transformers

Vision Transformers divide an image into patches and use self-attention to learn relationships between different regions.

This is useful when an obstacle is very small or sparse because the model can use information from the surrounding scene to understand the object.

However, high-resolution images create many image patches, increasing computational and memory requirements.

Conclusion

For a real-time autonomous vehicle with limited computational resources, a lightweight CNN-based detector is generally the practical choice.

Vision Transformers can provide strong performance when sufficient computational resources and training data are available. A hybrid CNN-Transformer model can also combine CNN local feature extraction with Transformer global-context modeling.

3. **Suggest a real-time inference pipeline for obstacle detection, considering computational constraints in embedded systems.**

Answer :

A suitable real-time pipeline is:

Step 1: Image Acquisition

The vehicle camera continuously captures road images or video frames.

Step 2: Preprocessing

Perform:

- Image resizing
- Normalization
- Noise reduction
- Region-of-interest selection

Instead of processing the complete high-resolution image at full size, the image can be resized to a suitable resolution.

Step 3: Lightweight Object Detection

Use an optimized detector such as a lightweight YOLO or MobileNet-based object detector.

The detector identifies:

- Obstacle location
- Bounding box
- Object class
- Confidence score

Example:

Step 4: Confidence Filtering

Remove detections having confidence below a predefined threshold.

For example:

Step 5: Non-Maximum Suppression

If multiple bounding boxes detect the same object, Non-Maximum Suppression (NMS) keeps the box with the highest confidence.

Step 6: Object Tracking

A tracking algorithm such as SORT or a similar lightweight tracker can track detected obstacles across consecutive frames.

This avoids performing expensive processing unnecessarily for every object in every frame.

Step 7: Distance and Collision Estimation

Use:

- Bounding-box size
- Stereo/depth camera
- LiDAR
- Radar

to estimate obstacle distance.

The system can calculate whether an obstacle is approaching the vehicle.

Step 8: Decision and Control

If an obstacle is detected within a dangerous distance:

Computational Optimization

For embedded systems:

- Use lightweight CNN architectures.
- Reduce input resolution when possible.
- Use model quantization such as INT8.
- Apply pruning and knowledge distillation.
- Use GPU/NPU acceleration.
- Process only relevant regions.
- Use batch size = 1 for real-time inference.
- Maintain a suitable frame rate.

Thus, the complete system provides fast obstacle detection with low latency and reduced memory/computational requirements, making it suitable for real-time autonomous driving.